
EEE5776. Applied Machine Learning



Module 2. Supervised Learning and Experimental Design

Machine Learning Pipeline

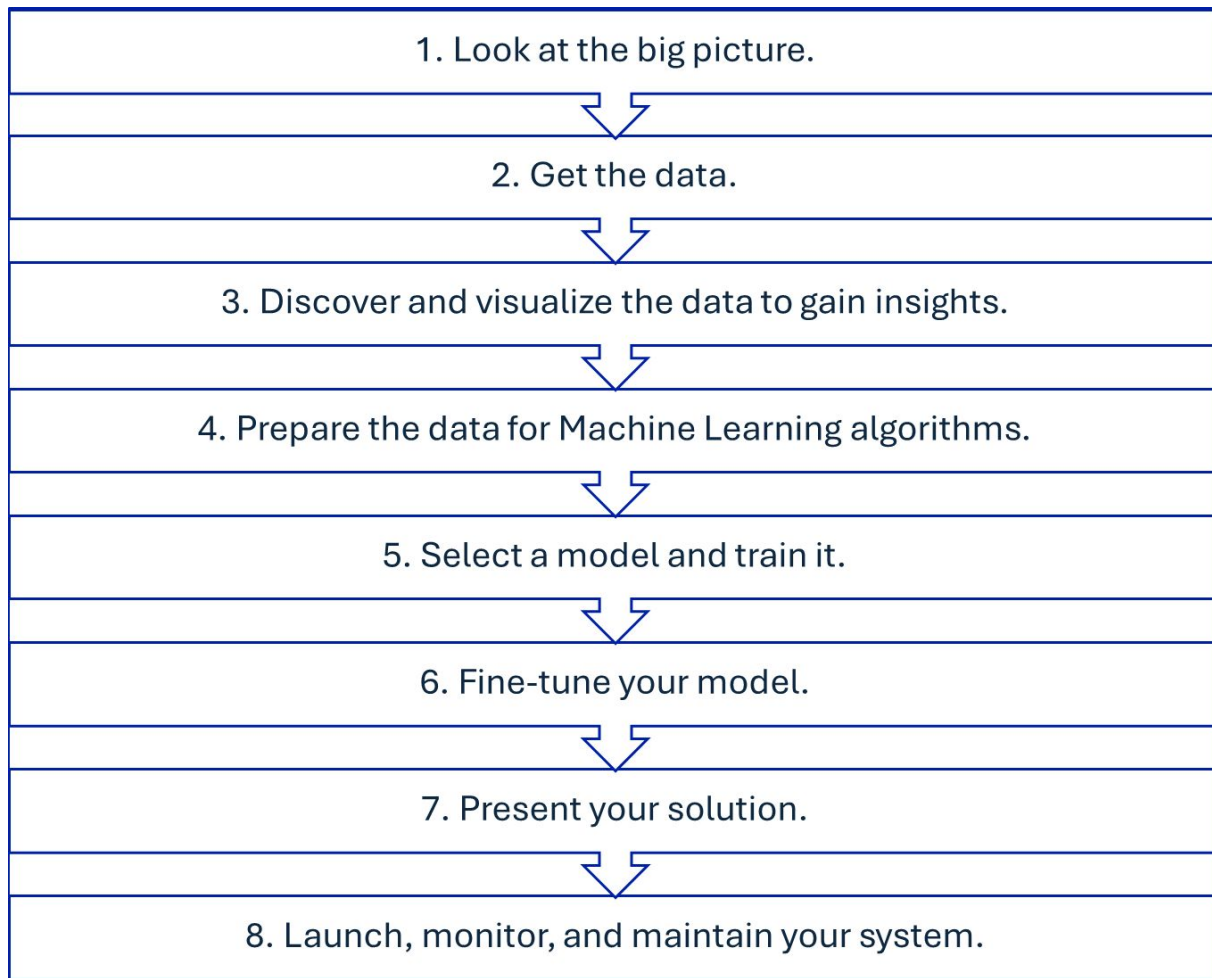
Regression Tasks

Hands-on Lab

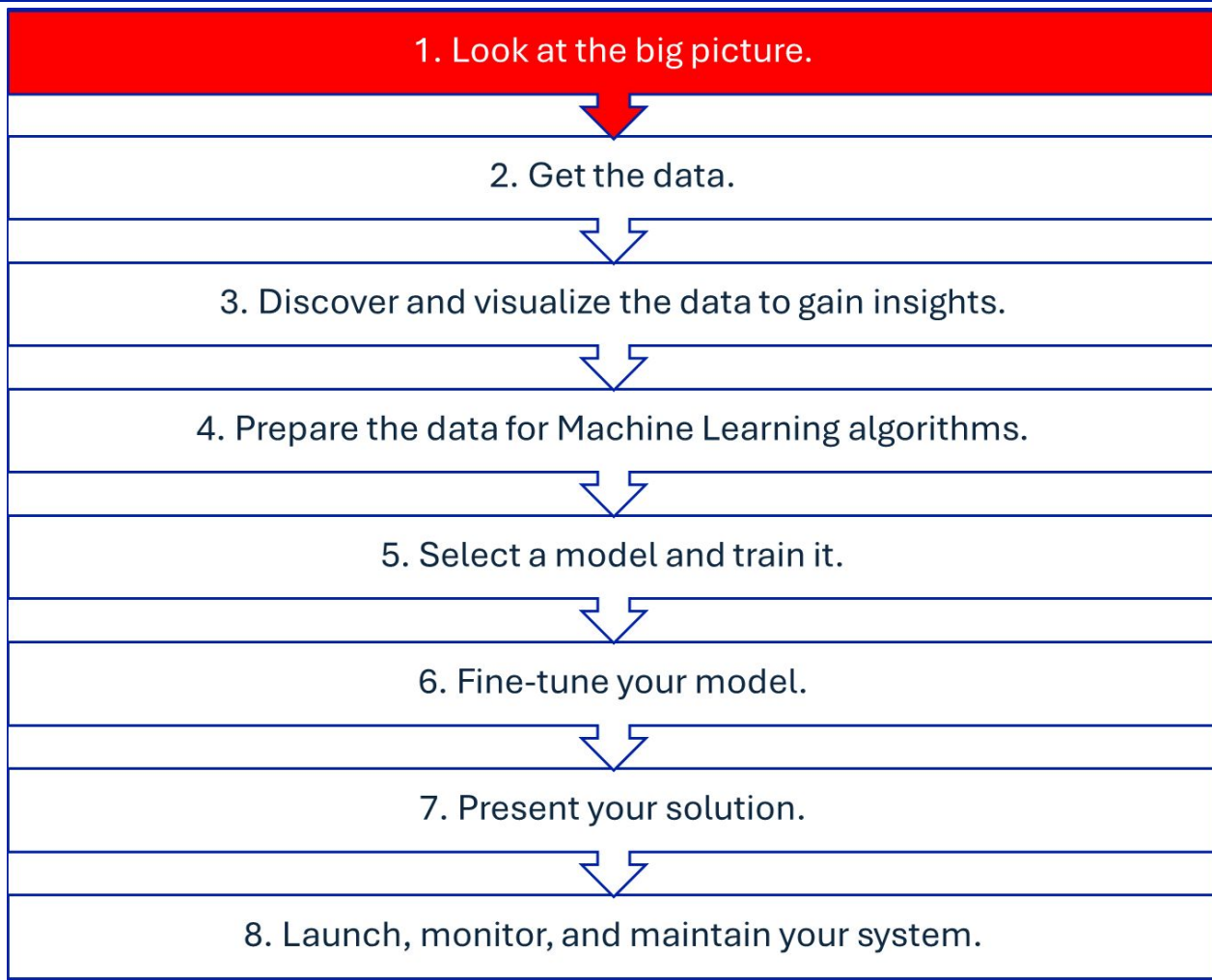


The Machine Learning Pipeline

End-to-End Machine Learning Process



End-to-End Machine Learning Process





Regression Tasks

Supervised Learning

Analogy time!

- Supervised learning is like studying vocabulary with flashcards. Each card shows a word (**input**) on one side and its meaning (**label**) on the other. Over time, the learner associates words with their meanings and can understand them in new contexts

Supervised Learning Tasks



Supervised Learning - Regression

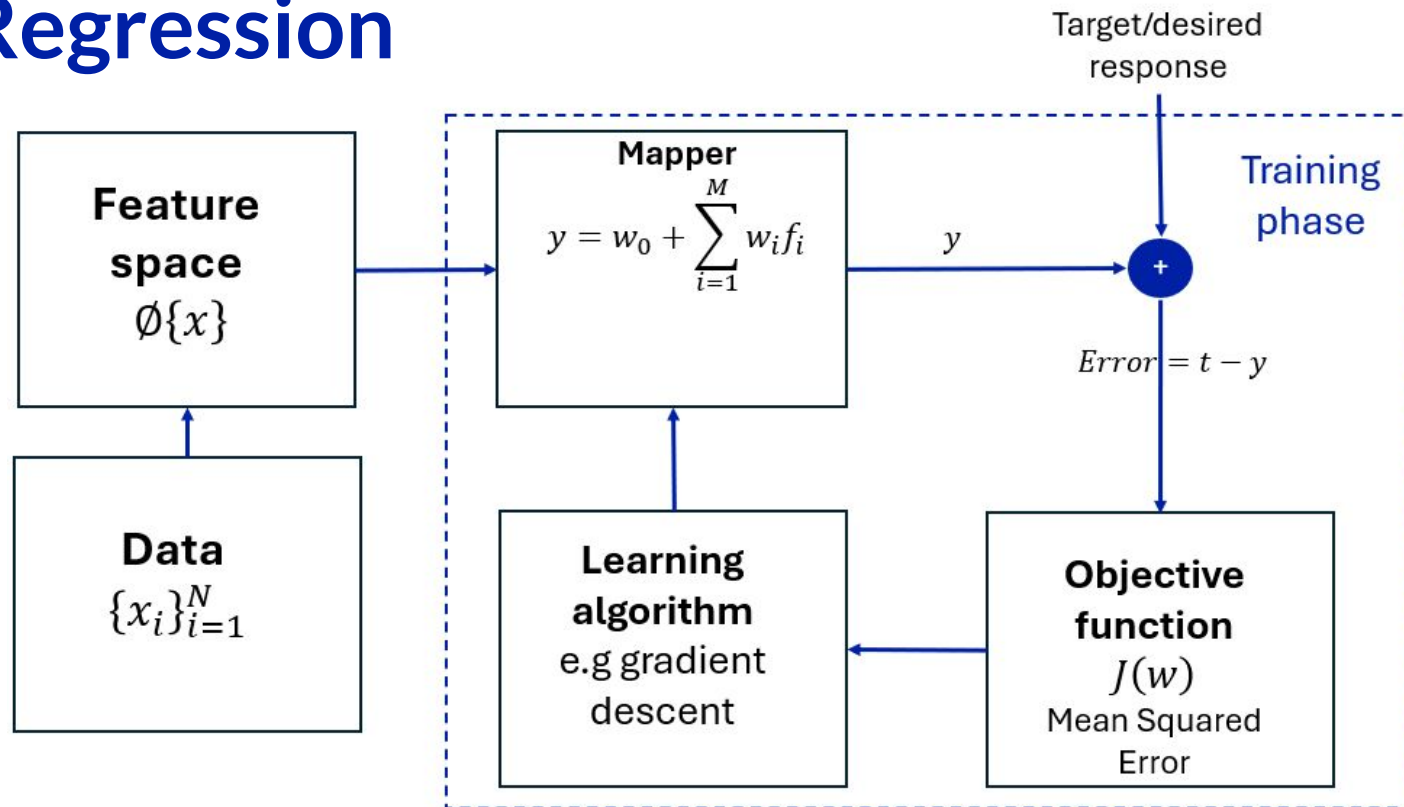
1. Look at the big picture: Predicting House Prices in California Using a Public Dataset

Features:

MedInc', 'HouseAge', 'AveRooms', 'AveBedrms',
 'Population', 'AveOccup', 'Latitude',
 'Longitude'



Supervised Learning - Multiple Linear Regression



Supervised Learning - Regression

Example

Let's work with this inputs

Median income of \$60k
 Average house age of 20 years
 Average 5 rooms per house
 Average 1 bedroom per house
 1000 people in the block
 Average 3 occupants per household
 Latitude: 34
 Longitude: -118

What is your price prediction?

Supervised Learning - Regression

Prediction

Predicted Median House Value for Input Data: \$292178.36

Model Evaluation:

Mean Squared Error: 0.5558915986952444

R² Score: 0.5757877060324508

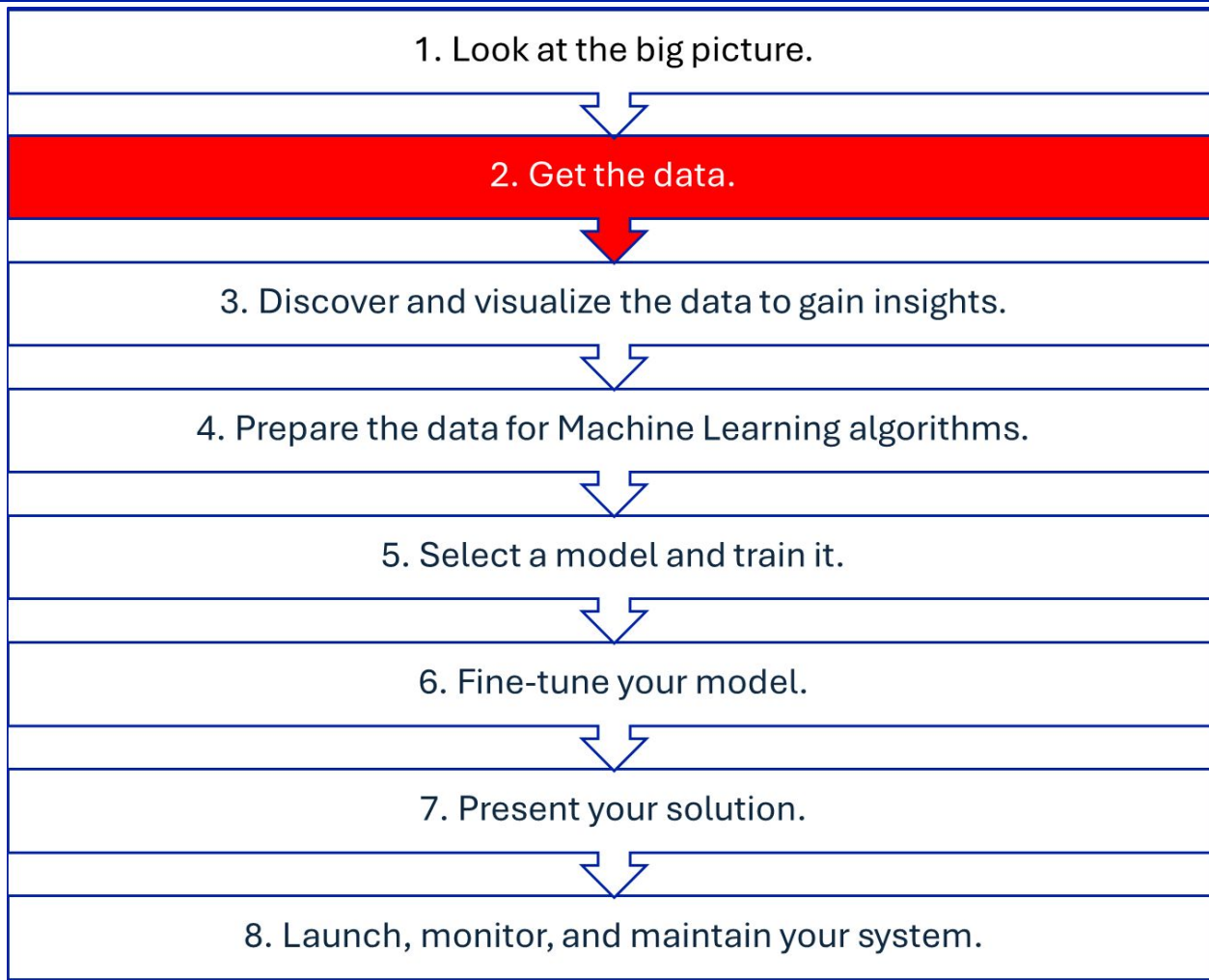
Feature Importances (from model coefficients):

Average Bedrooms	0.783145
Median Income (in \$10k)	0.448675
Longitude	0.433708
Latitude	0.419792
Average Rooms	0.123323
House Age (in years)	0.009724
Average Occupancy	0.003526
Population in Block	0.000002
dtype: float64	



The Machine Learning Pipeline

End-to-End Machine Learning Process



2. Get the Data

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('bmh')
import pandas as pd

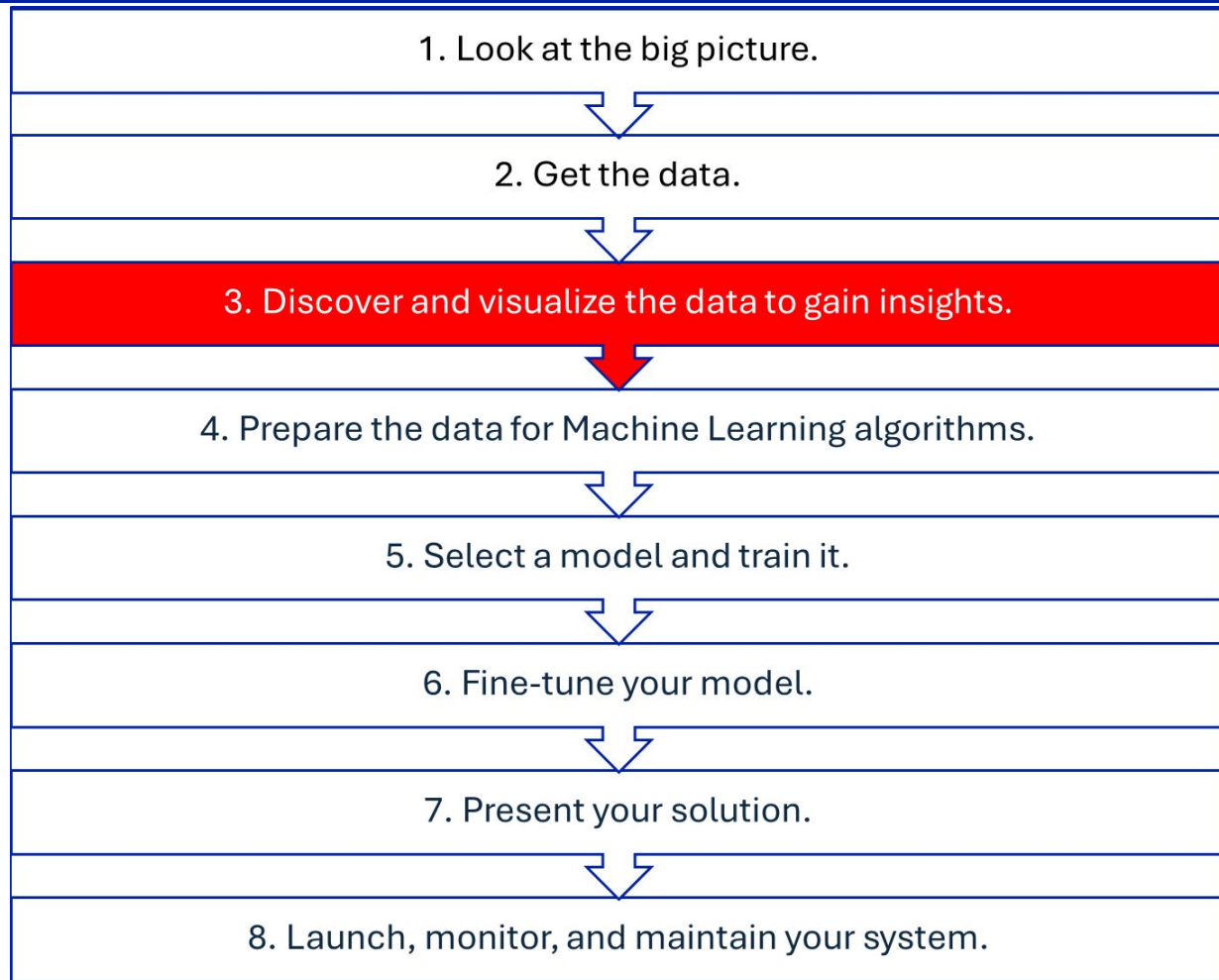
housing = pd.read_csv('housing.csv')
housing
```

2. Get the Data

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
0	-122.23	37.88	41.0	880.0	129.0
1	-122.22	37.86	21.0	7099.0	1106.0
2	-122.24	37.85	52.0	1467.0	190.0
3	-122.25	37.85	52.0	1274.0	235.0
4	-122.25	37.85	52.0	1627.0	280.0
...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0
20636	-121.21	39.49	18.0	697.0	150.0
20637	-121.22	39.43	17.0	2254.0	485.0
20638	-121.32	39.43	18.0	1860.0	409.0
20639	-121.24	39.37	16.0	2785.0	616.0

20640 rows × 10 columns

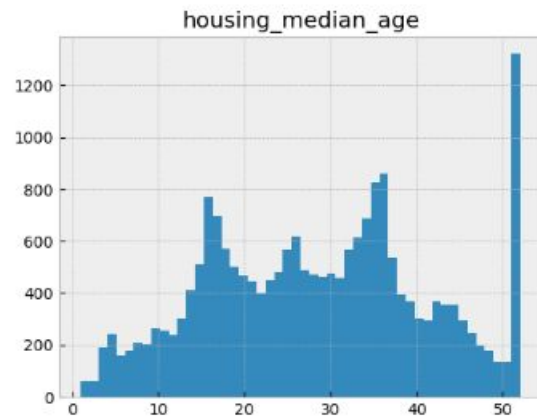
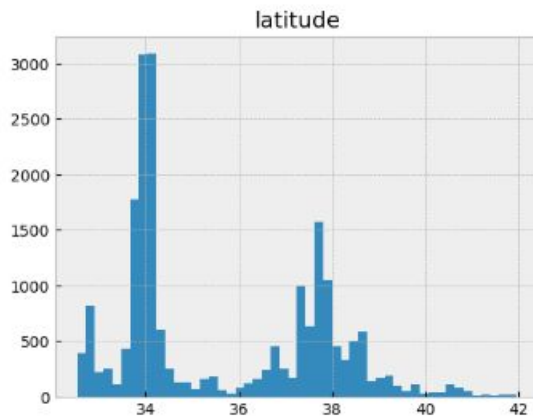
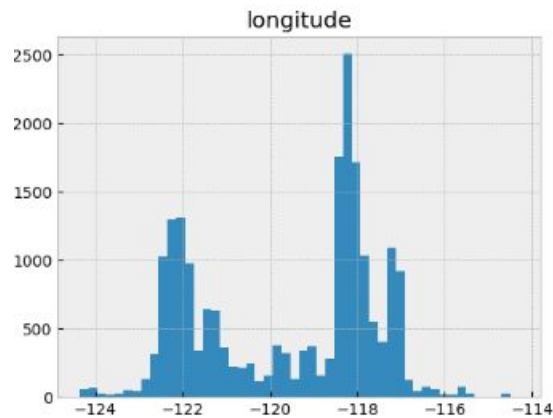
End-to-End Machine Learning Process



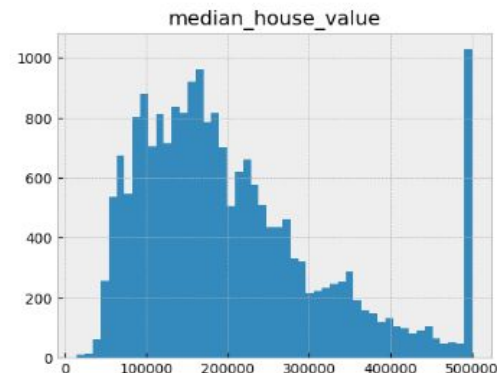
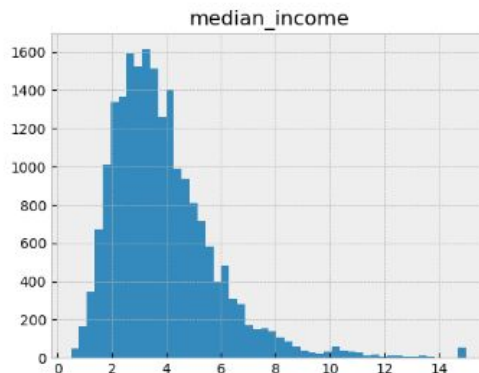
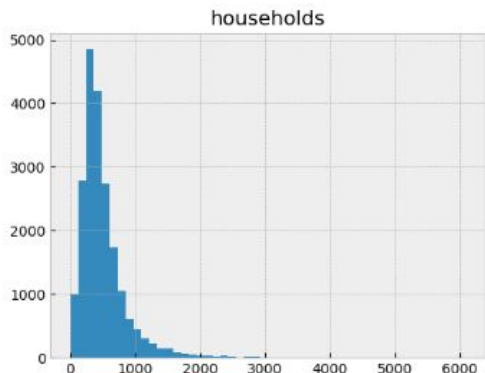
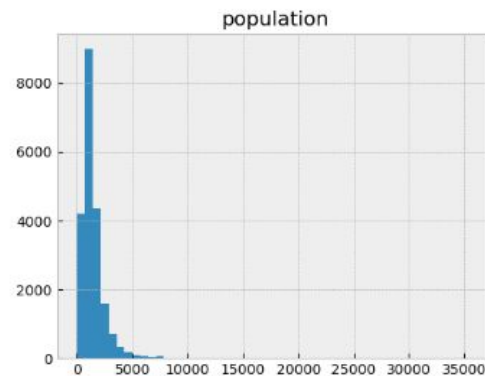
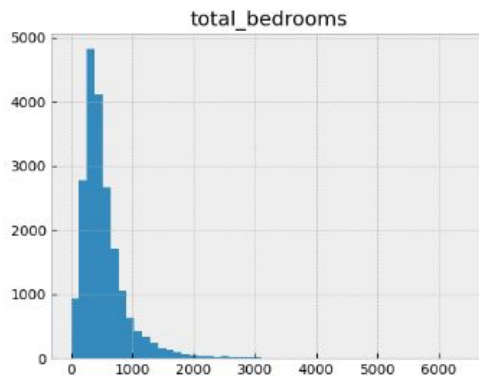
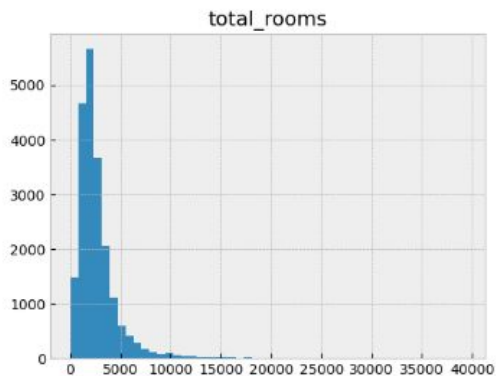
Step 3. Discover and Visualize the Data

A **histogram** is a good choice for numerical attributes. It shows the number of instances (on the vertical axis) that have a given value range (on the horizontal axis)

```
housing.hist(bins=50,figsize=(20,15));
```



Step 3. Discover and Visualize the Data



Step 3. Discover and Visualize the Data

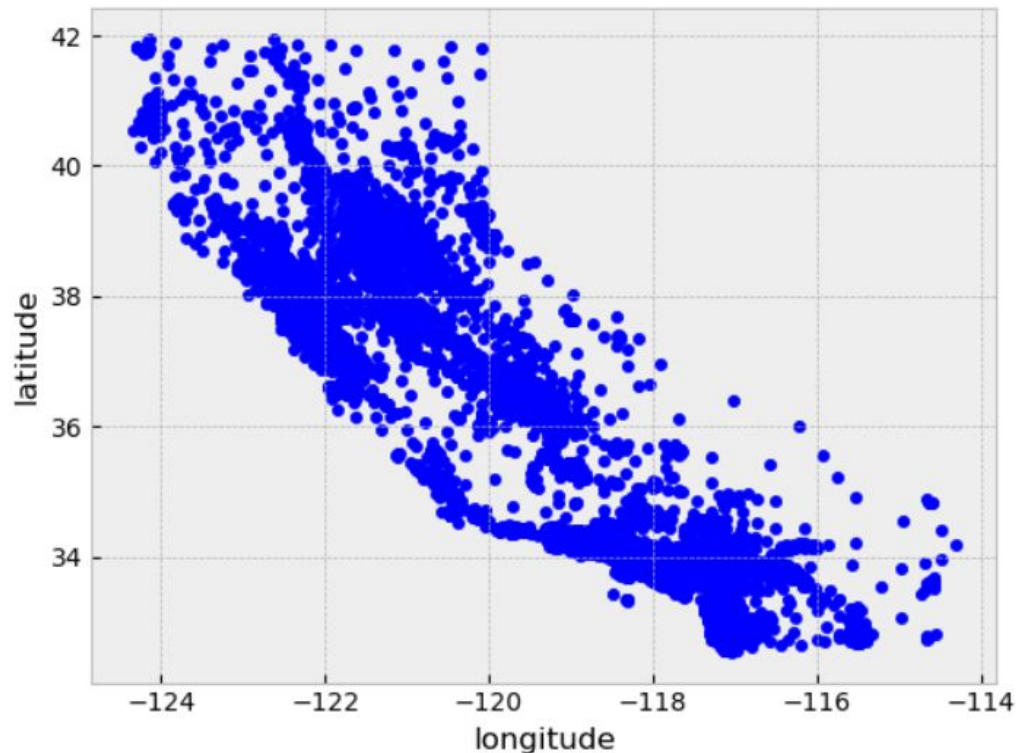
What can you observe?

1. First, the **median_income** attribute does not look like it is expressed in US dollars (USD).
2. The **housing_median_age** and the **median_house_value** were also capped.
3. All attributes have very different scales.
4. Many attributes have tail-heavy distributions. This may pose a challenge and we will see later how to transform them close to bell-shaped distributions.

Step 3. Discover and Visualize the Data

Let us look at
Geographical
Data

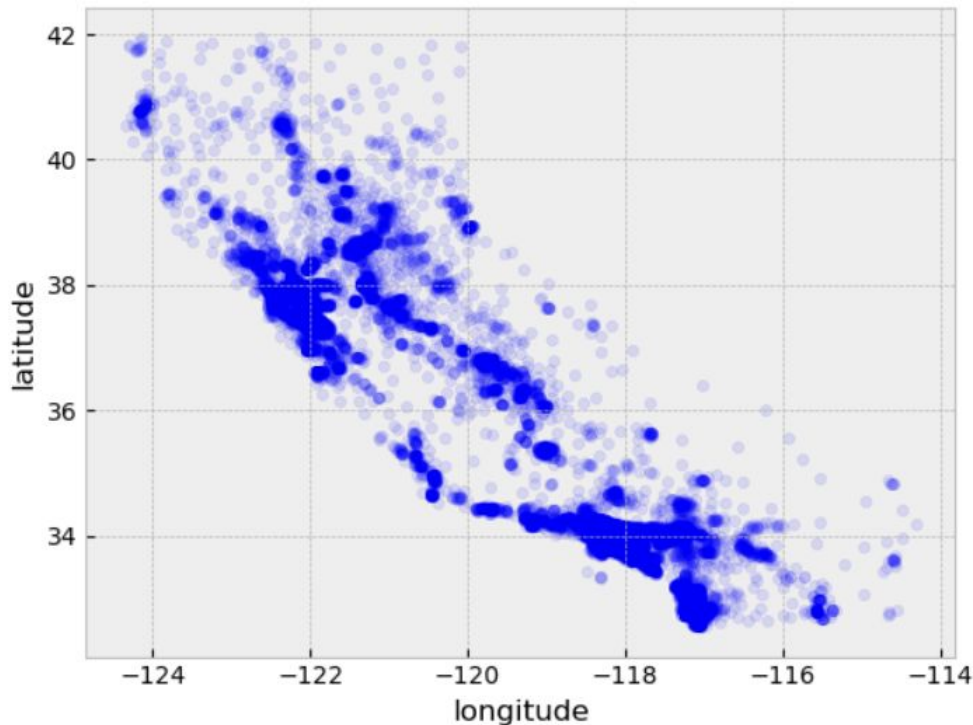
```
housing.plot(kind='scatter', x='longitude', y='latitude');
```



Step 3. Discover and Visualize the Data

Let us modify
the
transparency

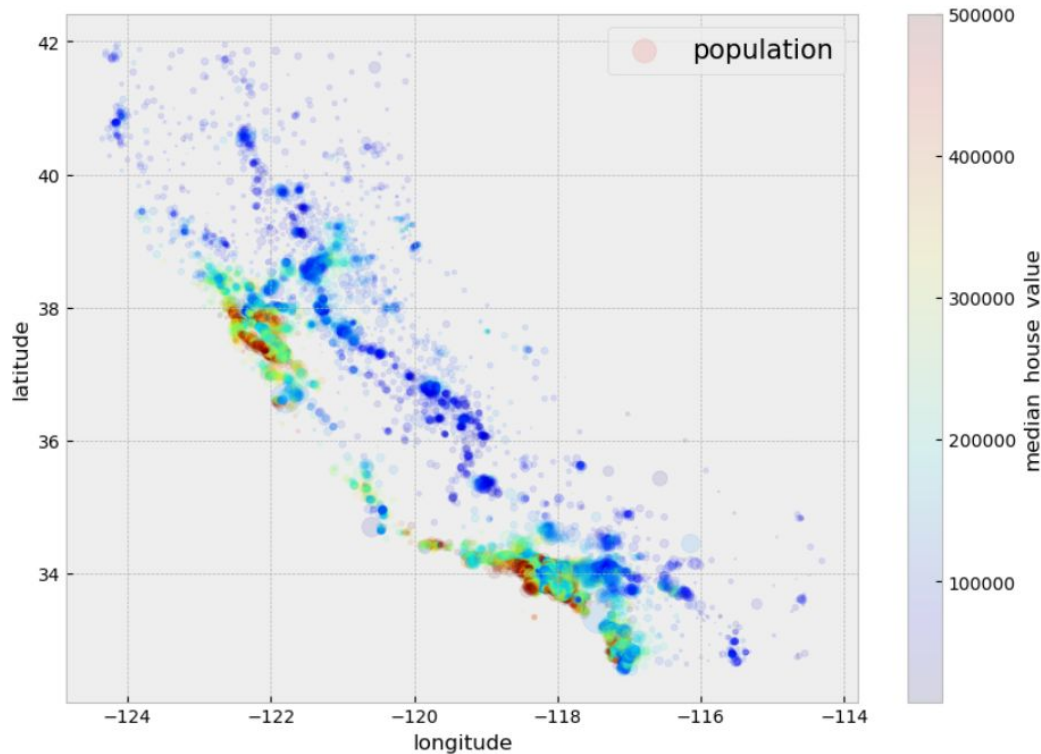
```
housing.plot(kind='scatter', x='longitude', y='latitude', alpha=0.1);  
  
#alpha controls transparency of points
```



Step 3. Discover and Visualize the Data

Let us add
house prices

```
housing.plot(kind='scatter', x='longitude', y='latitude', alpha=0.1,
             s=housing['population']/100, label='population',
             figsize=(10,7), c='median_house_value',
             cmap=plt.cm.jet, colorbar=True);
plt.legend(fontsize=15);
```



Step 3. Discover and Visualize the Data

We can compute the (linear) **Pearson's correlation coefficient** between pairs of attributes with the method **corr()**:

```
corr_matrix = housing.corr(method='pearson', numeric_only=True)
corr_matrix
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
longitude	1.000000	-0.924664	-0.108197	0.044568	0.069608
latitude	-0.924664	1.000000	0.011173	-0.036100	-0.066983
housing_median_age	-0.108197	0.011173	1.000000	-0.361262	-0.320451
total_rooms	0.044568	-0.036100	-0.361262	1.000000	0.930380
total_bedrooms	0.069608	-0.066983	-0.320451	0.930380	1.000000
population	0.099773	-0.108785	-0.296244	0.857126	0.877747
households	0.055310	-0.071035	-0.302916	0.918484	0.979728
median_income	-0.015176	-0.079809	-0.119034	0.198050	-0.007723
median_house_value	-0.045967	-0.144160	0.105623	0.134153	0.049686

...

Step 3. Discover and Visualize the Data

Let's check which attribute has the largest correlation with the dependent attributes (the labels):

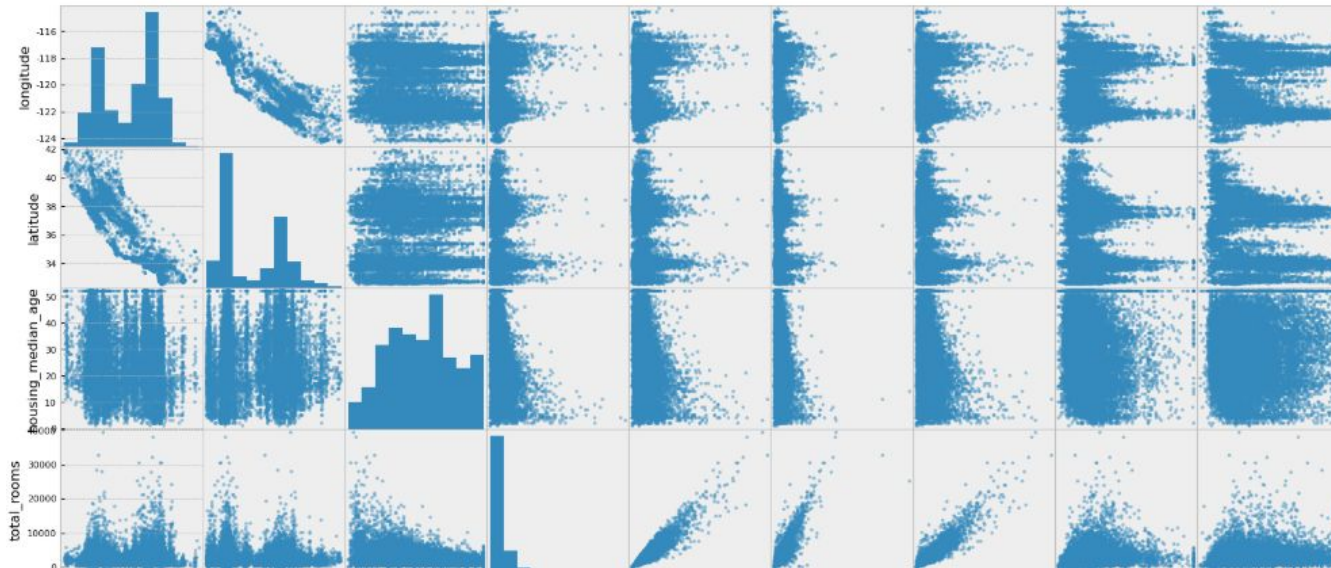
```
corr_matrix['median_house_value'].sort_values(ascending=False)
```

```
median_house_value    1.000000
median_income         0.688075
total_rooms           0.134153
housing_median_age    0.105623
households            0.065843
total_bedrooms        0.049686
population            -0.024650
longitude             -0.045967
latitude              -0.144160
Name: median_house_value, dtype: float64
```

Step 3. Discover and Visualize the Data

Another way to check for function, `scatter_matrix()` correlation between attributes is to use the pandas which plots every numerical attribute against every other numerical attribute:

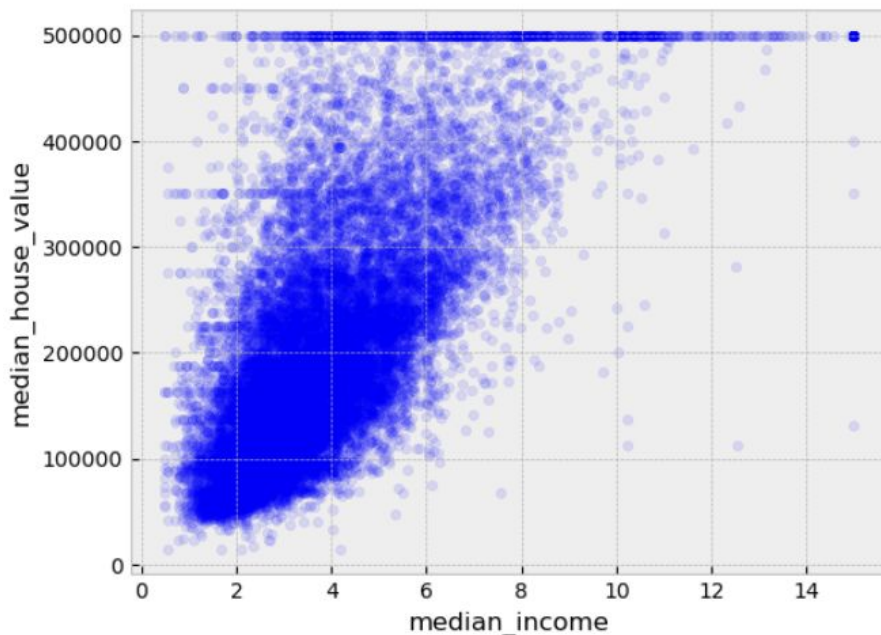
```
from pandas.plotting import scatter_matrix
scatter_matrix(housing, figsize=(20,20));
```



Step 3. Discover and Visualize the Data

The most promising attribute to predict the median house value is the **median_income**, so let's zoom in on their correlation scatterplot

```
housing.plot(kind='scatter', x='median_income', y='median_house_value', alpha=0.1)
```



Step 3. Discover and Visualize the Data

Instead of working with **pandas** dataframes, you can cast it first to a **numpy** array and then use appropriate indexing to access the corresponding column

```
housing_np = housing.to_numpy()
```

```
housing_np.shape
```

```
(20640, 10)
```

```
housing_np
```

```
array([[-122.23, 37.88, 41.0, ..., 8.3252, 452600.0, 'NEAR BAY'],
       [-122.22, 37.86, 21.0, ..., 8.3014, 358500.0, 'NEAR BAY'],
       [-122.24, 37.85, 52.0, ..., 7.2574, 352100.0, 'NEAR BAY'],
       ...,
       [-121.22, 39.43, 17.0, ..., 1.7, 92300.0, 'INLAND'],
       [-121.32, 39.43, 18.0, ..., 1.8672, 84700.0, 'INLAND'],
       [-121.24, 39.37, 16.0, ..., 2.3886, 89400.0, 'INLAND']],
      dtype=object)
```

```
housing.columns
```

```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
       'total_bedrooms', 'population', 'households', 'median_income',
       'median_house_value', 'ocean_proximity'],
      dtype='object')
```

```
plt.scatter(housing_np[:, -3], housing_np[:, -2], alpha=0.1)
plt.xlabel('median_income')
plt.ylabel('median_house_value');
```

Step 3. Discover and Visualize the Data

One last thing you may want to do before preparing the data for Machine Learning algorithms is to try out various **attribute combinations**

For example,

- the total number of rooms in a district is not very useful if you don't know how many households there are. What you really want is the number of rooms per household.
- Similarly, the total number of bedrooms by itself is not very useful: you probably want to compare it to the number of rooms.
- And the population per household also seems like an interesting attribute combination to look at. Let's create these new attributes

```
housing['rooms_per_household'] = housing['total_rooms']/housing['households']
housing['bedrooms_per_room'] = housing['total_bedrooms']/housing['total_rooms']
housing['population_per_household']=housing['population']/housing['households']
```

```
housing
```

Step 3. Discover and Visualize the Data

Now let's look at the correlation matrix again:

```
corr_matrix = housing.corr(method='pearson', numeric_only=True)
corr_matrix['median_house_value'].sort_values(ascending=False)
```

```
median_house_value    1.000000
median_income         0.688075
rooms_per_household   0.151948
total_rooms           0.134153
housing_median_age    0.105623
households            0.065843
total_bedrooms        0.049686
population_per_household -0.023737
population            -0.024650
longitude             -0.045967
latitude              -0.144160
bedrooms_per_room     -0.255880
Name: median_house_value, dtype: float64
```

1. The new feature **bedrooms_per_room** is much more correlated with the median house value than the total number of rooms or bedrooms. Since the correlation is negative, this means that houses with a lower bedroom/room ratio tend to be more expensive.
2. The number of rooms per household is also more informative than the total number of rooms - obviously, the larger the houses, the more expensive they are.



Hands-on Lab

Let's get started with code!

- Install **Git**. Alternatively, you can install **GitHub Desktop**.
- Create a **GitHub account**.
- How to **clone** a repository?
 - You can use Git Bash to clone a repo (alternative: use GitHub Desktop interface).
- Getting the **latest edits** from a repository with git pull.
 - To pull from a repository, simply call git pull using Git Bash.
- How to **manage files** within a repo?
 - The 3 most used Git commands are: git pull, git add, git commit and git push. You can call these commands directly with the Git Bash console or using the GitHub Desktop interface.
- Now let's practice. Start working INDIVIDUALLY in your **Lab 01** (Due Sunday, January 26th at 11:59pm). It is posted in Canvas under Module 2.

